Failure (a crashing tool, and a bad description) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `tool_selection_practice.py` (Failure section): this is two real production bugs you *will* cause yourself at least once — a tool that crashes (raises instead of returning), and a tool whose description is so vague the model calls it at the wrong moment. The exercise makes you create both on purpose, where nothing's at stake, and fix them: catch the exception and return a clean, model-readable `Error: ...` string, and sharpen the description so the model chooses correctly. **If not:** the Build Task's "one tool designed to sometimes fail" would be the first time you ever turned a raised exception into a clean, model-readable error string.

Solution

```
# tool_selection_practice.py — Failure section
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


# --- Half 1: a crashing tool, made safe, and the error sent back ---
@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b. Returns an error message string if b is zero."""
    try:
        return str(a / b)
    except ZeroDivisionError as e:
        return f"Error: {e}"


model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
model = model.bind_tools([flaky_divide])

# The same growing message list as Doc04's memory exercise.
messages = [{"role": "user", "content": "Use the tool to divide 10 by 0"}]
response = model.invoke(messages)
if not response.tool_calls:
    print(f"No tool called: {response.content}")
else:
```

```
    messages.append(response)          # the model's request goes in first
    for call in response.tool_calls:
        result = flaky_divide.invoke(call["args"])
        print(f"Tool call {call['args']} -> {result}")
        # tool_call_id ties this result to the exact request it answers
        messages.append(
            {"role": "tool", "content": result, "tool_call_id": call["id"]}
        )
    # a second call, now with the error in history — the model reacts to it
    final = model.invoke(messages)
    print(f"Model's reaction to the error: {final.content}")


# --- Half 2: a vague description, sharpened ---
# Both tools share the name "get_weather", so the description is the only
# thing that differs between the two trials.
@tool("get_weather")
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"


@tool("get_weather")
def precise_weather(city: str) -> str:
    """Get the current weather (temperature, conditions) for a named city."""
    return f"Sunny in {city}"


# Count how often the model calls a given weather tool over 5 runs.
def count_tool_calls(weather_tool, prompt, runs=5):
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([weather_tool])
    called = 0
    for run_number in range(runs):
        response = model_with_tools.invoke(prompt)
        if response.tool_calls:
            called = called + 1
    return called


print()
prompt = "what's the weather like"
vague_hits = count_tool_calls(vague_weather, prompt)
precise_hits = count_tool_calls(precise_weather, prompt)
print(f"Vague description:   called {vague_hits}/5 times")
print(f"Precise description: called {precise_hits}/5 times")
```

Expected output (example):

```
Tool call {'a': 10, 'b': 0} -> Error: division by zero
Model's reaction to the error: Dividing by zero isn't possible.

Vague description: called 1/5 times
Precise description: called 5/5 times
```